

From netlist to verified model

Automatic real-number-model and testbench generation
for mixed-signal verification

Abstract

Real number models are how a mixed-signal block joins a digital regression suite — and writing one by hand is slow, keeping it faithful as the circuit changes is slower, and proving it is faithful is the part that usually never happens. **spice2rnm** closes the loop in one command: give it a SPICE netlist and it delivers a SystemVerilog real-number model, a UVM-MS verification environment, and the evidence that both agree with the circuit.

Three properties set it apart. Every golden value in every generated testbench is a *SPICE measurement* of your circuit, never the model's own equations. Every generated environment is proven able to fail, by injecting faults into the model and confirming the verdict flips. And the verification strategy adapts to what each block is *for* — a filter is checked as a frequency response, a duty-cycle corrector by where its edges land, a phase interpolator by phase per code.

This paper presents three case studies of increasing difficulty, with results produced end to end by the shipped tool in under six minutes on a laptop.

Contents

1	The problem	1
2	What you get from one command	2
3	Case study 1 — Two-pole filter	2
4	Case study 2 — Duty-cycle corrector	3
4.1	A model that follows the circuit's dynamics	3
4.2	Verified by function, not by formula	3
5	Case study 3 — Phase interpolator	3
6	Results at a glance	4
7	Why a green testbench is worth believing	4
8	Functional specifications as input	4
9	AI where it helps, physics where it decides	5
10	Scope and requirements	5

1 The problem

A behavioural model has to be two contradictory things: fast enough to run in a digital regression, and faithful enough that passing it means something. Hand-written models drift from their circuits, and the drift is silent — the model still compiles, still simulates, still passes.

The usual defence is a hand-written testbench, which has the same problem one level up. Somebody decides what to check and how close is close enough, and those decisions are rarely written down next to the numbers they produced. When the circuit is revised, the model, the testbench, and the tolerances all have to be revisited by hand — so in practice they are not.

spice2rnm regenerates all three from the netlist, on demand. The model, the environment that checks it, and the evidence trail are outputs of one command, so they can never fall out of step with the circuit or with each other.

2 What you get from one command

```
spice2rnm dcc.cir --input-node clk --output-node vout \
--emit-uvm-ms --spec dcc_spec.md
```

- **A SystemVerilog real-number model**, plain **real** ports, simulator-portable, with the model structure chosen automatically to match the block’s measured behaviour — including blocks whose dynamics move with signal level, where any fixed-response model is wrong across most of its range. The model’s own ports are plain **real**, so it runs on any IEEE 1800 simulator with no nettype or AMS support at all; the generated DMS wrapper is what lifts it onto the testbench’s nettypes.
- **A UVM-MS verification environment** following the Accellera UVM-MS specification, built on IEEE 1800 — not on Verilog-AMS. Analog values travel on *user-defined nettypes* (IEEE 1800-2017 §6.6.7) with a generated resolution function, so multiple drivers on a net — an offset source, a noise injector, a second bridge — combine in the net type rather than in hand-edited testbench code; there is no **wreal** anywhere, and no AMS licence needed to run it. Bridge and wrapper ports are **interconnect** (§6.6.8), inheriting the testbench’s nettype instead of naming one, and scoreboard golden values are SPICE measurements of your circuit.
- **Equivalence evidence**: circuit and model driven with the identical stimulus, compared in the domain that matters for the block — waveform error for signal-path blocks, edge placement for clock-path blocks — with the aligned traces archived so any verdict can be re-derived from its own run directory.

An optional plain-language functional specification supplies what no netlist can: the block’s intended operating range and its acceptance criteria (Section 8).

3 Case study 1 — Two-pole filter

A two-pole RC low-pass: the representative signal-path case. The generated model is checked four independent ways.

Chirp equivalence	0.0290 against a 0.15 threshold	PASS
PRBS (independent stimulus)	0.0097 normalised	PASS
UVM-MS, DC section	51 checks, 0 failed	
UVM-MS, AC section	19 checks, 0 failed	
worst magnitude	0.0390 dB of a 0.25 dB tolerance	
worst phase	2.70° of an 8.00° tolerance	

The PRBS row matters more than it looks: the model is fitted against a chirp and then checked against a pseudo-random sequence it never saw. A model that had merely memorised its characterisation stimulus would be exposed here.

4 Case study 2 — Duty-cycle corrector

A comparator-based duty-cycle corrector whose dominant pole moves by a factor of 613,666 across its input range. Blocks like this defeat fixed-response modelling outright, and they defeat conventional AC-based verification too — which is exactly why they are usually the blocks whose hand-written models are trusted least.

4.1 A model that follows the circuit’s dynamics

spice2rnm detects the level dependence during characterisation and builds a model whose dynamics are *scheduled against measured large-signal behaviour* — including the circuit’s rise/fall timing asymmetry, which it measures at 95 ps and reproduces. At a 50% input duty the model matches the circuit’s output duty to 0.000 percentage points.

Transient equivalence	0.0769 against a 0.15 threshold	PASS
UVM-MS, DC section	51 checks, 0 failed	
UVM-MS, duty section	15 checks, 0 failed	
worst duty error	0.314 pp of a 0.500 pp tolerance	

4.2 Verified by function, not by formula

The generated environment for this block contains *no AC section* — by measurement, not by omission. A small-signal sweep linearises the circuit at one operating point, and a comparator in normal operation is never near that regime; checked that way, a demonstrably accurate model fails every point by more than 55 dB, telling you nothing about the model and everything about the mismatch between the check and the block.

Instead, the environment checks what a duty-cycle corrector is *for*. The tool drives the circuit with clocks at multiple rates and input duties, records the output duty from SPICE, and holds the model to those goldens — a table spanning 20 percentage points of measured behaviour, so a model that ignored its input could not reproduce it.

5 Case study 3 — Phase interpolator

A thermometer-coded phase interpolator has essentially no amplitude transfer at all — there is no frequency response worth fitting, and a voltage-domain comparison scores noise. Conventional flows stop here.

spice2rnm recognises the block’s function and switches strategy: it measures phase per code, generates a delay-line model, and emits a *phase* verification environment — one measured golden per control code, no DC or AC section, because neither describes edge placement.

Codes measured	9 of 9
Total traversal	−500.1 ps, monotonic
Mean step (1 LSB)	−62.51 ps
Worst DNL	−100.5 ps (−1.61 LSB)
UVM-MS phase section	9 checks, 0 failed
worst	3.26 ps of a 6.25 ps tolerance

The tolerance is a tenth of an LSB, at the model’s own quantisation floor — tight enough to fail on a real regression, never on rounding. The DNL figure is a property of the circuit, reported faithfully; the model’s job is to reproduce it, and it does.

6 Results at a glance

Case study	Environment	Checks	Failed
Filter	DC + AC	51 + 19	0
Duty corrector	DC + duty	51 + 15	0
Phase interpolator	phase per code	9	0

All three case studies — characterisation, model generation, both simulators, and every generated environment — run end to end in 355 seconds on a laptop. The generated artifacts themselves — the models, the complete environments, and the evidence files, exactly as the tool wrote them — are published for inspection at

<https://github.com/vrajeshprakhya/spice2rnm-demos>

under `showcase/`, stamped with the tool version and run summary that produced them.

7 Why a green testbench is worth believing

Generated verification has a credibility problem: if the same tool writes the model and the test, what stops it from marking its own homework? Three design rules.

Goldens are measurements, never the model. Every golden value in every generated environment is a SPICE measurement of your circuit. Checking a model against the equations it implements would pass unconditionally; spice2rnm never does it.

Every environment is proven able to fail. The tool injects faults into the generated model and confirms the verdict flips — and the detection floor is *measured*, not asserted. For the duty-cycle corrector’s environment:

Injected fault	Detected by
comparator offset, 20 mV	DC and duty sections
dynamics slowed 10×	duty section (DC is structurally blind)
dynamics slowed 50×	duty section (DC is structurally blind)

The middle column is the point: settled-DC checks — the whole of what many hand-written environments test — cannot see *any* error in a block’s dynamics. The function-matched sections exist because of that blindness, and the fault sweep is re-run against the shipped model so the detection claims stay current.

Sensitivity is pinned from both sides. A check that fails everything is as useless as one that passes everything, so fault injection also confirms that errors *within* tolerance do not trip the environment.

8 Functional specifications as input

A netlist says what a circuit *is*. It never says what it is *for*, what rate it must run at, or how close is close enough. Hand those to the tool as a plain-language document — a datasheet excerpt, a requirements page — and it reads out the facts that shape verification:

From the specification	Used for
operating clock range	where the environment drives the block
accepted input duty range	the stimulus sweep
duty accuracy requirement	the pass/fail tolerance
block function	which environment is generated

On the duty-cycle corrector, the specification-driven run checks 35–65% input duty at the specified clock rates against the specification’s own 1.0 pp requirement: 15 checks, 0 failed, worst 0.428 pp. **PASS**

The specification is cross-checked, never trusted. Every extracted fact is validated against what the circuit was actually measured to do, and the measurement wins. When the two disagree, that disagreement is surfaced as a finding:

```
SPEC CONFLICT: the spec says this block operates up to 4e+08 Hz, but
the fitted model is only valid below 1.195e+08 Hz -- 3.3x lower.
```

That is a coverage gap between a requirement and a characterisation, caught at model-generation time instead of at silicon correlation — and it is the kind of finding no amount of manual model review produces, because no one document contains both halves of it.

9 AI where it helps, physics where it decides

spice2rnm uses language models at three points: proposing block boundaries in the device graph, classifying what a block is for, and reading functional specifications. All three follow one rule: **intent from the model, physics from the measurement**. Every proposal is validated against the measured circuit before it is used, and a proposal that fails validation is discarded — the run continues on measured defaults, and says so. Without an API key, every AI feature declines cleanly and the pipeline runs entirely on measurement.

10 Scope and requirements

- **Inputs:** SPICE netlists (ngspice dialect), flat or with subcircuits; optional plain-language functional specification.
- **Outputs:** SystemVerilog real-number models (plain **real** ports), UVM-MS 1.0 environments, equivalence reports with archived waveforms.
- **Toolchain:** ngspice for characterisation; any Icarus-compatible SystemVerilog simulator with UVM support for the generated environments.
- **Block-level scope:** single-input single-output blocks and compositions of them, including current-mode blocks sharing a node. Hierarchical composition trades some accuracy for structure; the equivalence report states the composed figure separately from the per-block figures, so the trade is visible, not hidden.

Every figure in this document was produced by the shipped tool from its bundled demonstration circuits, most recently on 2026-09-01. The demonstration suite at <https://github.com/vrajeshprakhya/spice2rnm-demos> regenerates them in one command.